

Full Stack Development with MERN Project

Documentation

1. Introduction

- Project Title: ShopEZ - Online E-Commerce Shopping Platform
- Team Members:

Name	Role
Buyya Vaishnavi	Frontend Development, Backend Development, Database Management, Testing

2. Project Overview

Purpose :

ShopEZ is a complete, full-stack e-commerce platform developed to simplify the process of browsing, buying and selling products online. Traditional offline shopping and scattered single-seller websites make it difficult for customers to compare products and for small sellers to reach a wider audience. ShopEZ provides a centralized marketplace where customers can search for products, add them to a cart, place orders and track order status, while sellers can list and manage their own products.

The platform provides dedicated interfaces for customers, sellers and administrators. Sellers can add, update and remove their product listings and monitor orders placed for their products, while administrators can oversee all users, products and orders on the platform. The system ensures secure communication and role-based access using JWT authentication.

Features :

- Customer Registration and Login
- Seller Registration and Product Listing
- Role-Based Accounts (Customer / Seller / Admin)
- Product Search and Filtering by Category
- Shopping Cart Management
- Order Placement and Checkout
- Order Status Tracking (Pending, Confirmed, Processing, Shipped, Delivered, Cancelled)
- Customer Dashboard, Seller Dashboard and Admin Dashboard
- Secure JWT Authentication
- Role-Based Access Control

3. Architecture

Frontend Architecture :

The frontend of ShopEZ is developed using React 19 with TypeScript, powered by Vite, and routed with TanStack Router. React components are responsible for creating the various UI screens such as the product listing, cart, checkout, and dashboards. TanStack Query and Axios handle data fetching and communication with the backend APIs. Styling is implemented with Tailwind CSS and shadcn/ui components. Each user can only access pages according to their designated role (customer, seller or admin) via protected routes.

Backend Architecture :

Node.js and Express.js (written in TypeScript) are used for the development of the backend. REST APIs handle authentication, product management, cart operations and order processing. Controllers handle the business logic while routes handle the incoming requests. JWT middleware validates user sessions on the server side and protects secured routes.

Database Architecture :

MongoDB is used as the database, accessed through Mongoose ODM. It stores data related to the system including user accounts, product listings, shopping carts and order details.

System flow:

1. The Customer and Seller register themselves by providing name, email and password (role selected as customer or seller).
2. The Seller adds product listings with name, description, price, category, stock and image.
3. The Customer searches or browses products by category and adds items to the shopping cart.
4. The Customer proceeds to checkout, provides a shipping address and places the order.
5. The Order status progresses from Pending → Confirmed → Processing → Shipped → Delivered, or can be Cancelled by the Customer before shipping.
6. The Admin monitors all users, products and orders through the Admin Dashboard.

4. Setup Instructions

Prerequisites

Before running the application, ensure the following software is installed:

- Node.js v18 or above
- npm or bun
- MongoDB (local, Docker, or Atlas cloud account)
- Git
- VS Code

Installation Steps:

Clone Repository:

```
git clone <repository-url>
```

Install Frontend Dependencies:

```
npm install
```

Install Backend Dependencies:

```
cd backend && npm install
```

Environment Variables:

Create .env in the backend folder:

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_secret_key
```

```
PORT=5000
```

5. Folder Structure

ShopEZ — Project Folder Structure

ShopEZ-Complete

├── src (Frontend - React 19 + TypeScript)

│ ├── components

│ │ ├── ProductCard.tsx

│ │ ├── SiteHeader.tsx

│ │ └── ui

│ ├── hooks

│ ├── lib

│ ├── routes (TanStack Router)

│ │ ├── _root.tsx

│ │ ├── index.tsx

│ │ ├── login.tsx

│ │ ├── register.tsx

│ │ ├── product.\$id.tsx

│ │ ├── cart.tsx

│ │ ├── checkout.tsx

│ │ ├── orders.tsx

│ │ ├── profile.tsx

│ │ ├── dashboard.tsx

│ │ └── about.tsx

│ ├── router.tsx

│ ├── server.ts

│ ├── start.ts

│ └── styles.css

├── backend (Express.js + TypeScript)

│ ├── config

│ ├── controllers

│ │ ├── authController.ts

│ │ ├── productController.ts

│ │ ├── cartController.ts

│ │ └── orderController.ts

│ ├── middleware

│ ├── models

│ │ ├── User.ts

│ │ ├── Product.ts

│ │ ├── Cart.ts

│ │ └── Order.ts

│ ├── routes

│ │ ├── auth.ts

│ │ ├── products.ts

│ │ ├── cart.ts

│ │ └── orders.ts

│ ├── seed.ts

│ ├── server.ts

│ └── package.json

├── public

├── docker-compose.yml

├── vite.config.ts

├── package.json

└── README.md

6. Running the Application

Start Backend

```
cd backend  
npm run dev
```

Backend URL:

<http://localhost:5000>

Start Frontend

```
npm run dev
```

Frontend URL:

<http://localhost:5173>

7. API Documentation

Authentication APIs

Register User POST /api/auth/register

Login User POST /api/auth/login

Get Profile GET /api/auth/profile

Update Profile PUT /api/auth/profile

Product APIs

Get All Products GET /api/products

Get by Category GET /api/products/category/:category

Get Product by ID GET /api/products/:id

Create Product (Seller) POST /api/products

Update Product PUT /api/products/:id

Delete Product DELETE /api/products/:id

Cart APIs

Get Cart GET /api/cart

Add to Cart POST /api/cart/add

Update Cart Item PUT /api/cart/update

Remove Item DELETE /api/cart/remove/:productId

Clear Cart DELETE /api/cart/clear

Order APIs

Create Order POST /api/orders

Get Orders GET /api/orders

Get Order by ID GET /api/orders/:id

Update Order Status PUT /api/orders/:id/status

Cancel Order PUT /api/orders/:id/cancel

8. Authentication

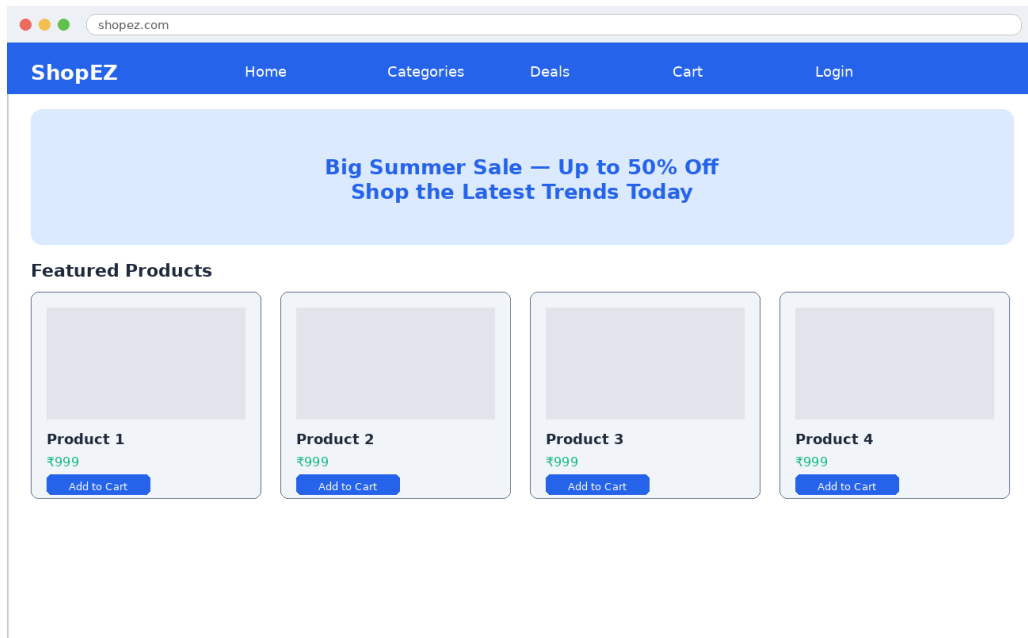
The ShopEZ application uses JWT (JSON Web Token) based authentication to protect user sessions. Once a user logs in successfully, a JWT token is issued and stored on the client-side. It is sent along with all future requests for authentication purposes.

Some security practices employed include:

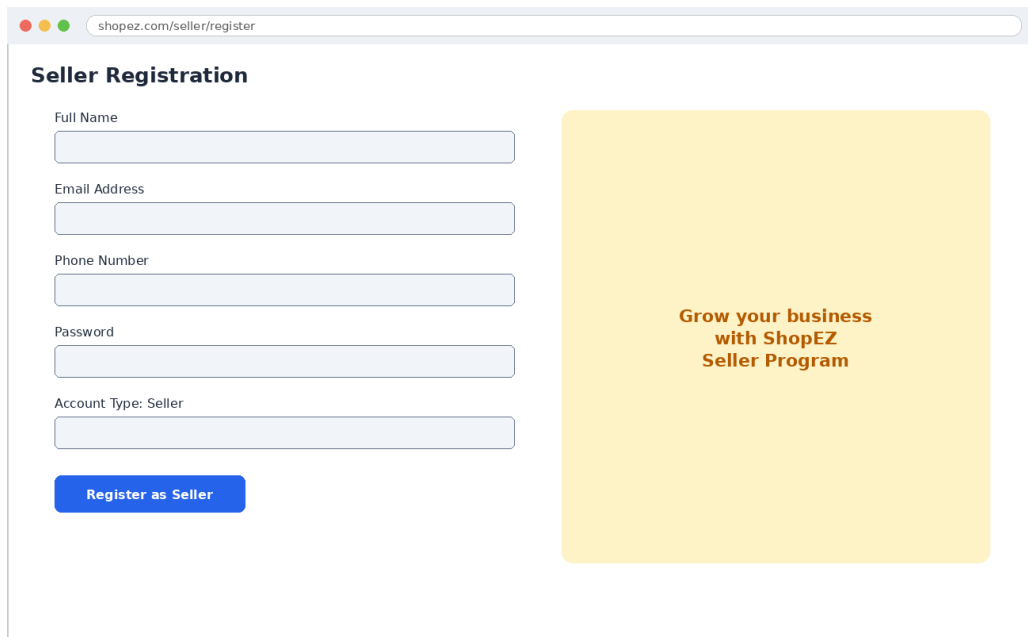
- Password hashing with bcryptjs
- JWT Authentication
- Protected routes
- Role-based access control
- Secure API validation

9. User Interface

Home Page :



Seller Registration :



The screenshot shows the ShopEZ Seller Registration Page. The browser address bar displays "shopez.com/seller/register". The page title is "Seller Registration". On the left, there is a registration form with the following fields: Full Name, Email Address, Phone Number, Password, and Account Type: Seller. A blue "Register as Seller" button is located below the form. On the right, there is a large yellow box with the text "Grow your business with ShopEZ Seller Program".

Seller Dashboard - Add Product :

shopez.com/dashboard/add-product

Seller Dashboard — Add New Product

Product Name

Description

Category

Price (Rs.) / Original Price (Rs.)

Stock Quantity

Product Image URL

Publish Product

Listed under seller's product catalog instantly

Admin Dashboard :

shopez.com/admin/dashboard

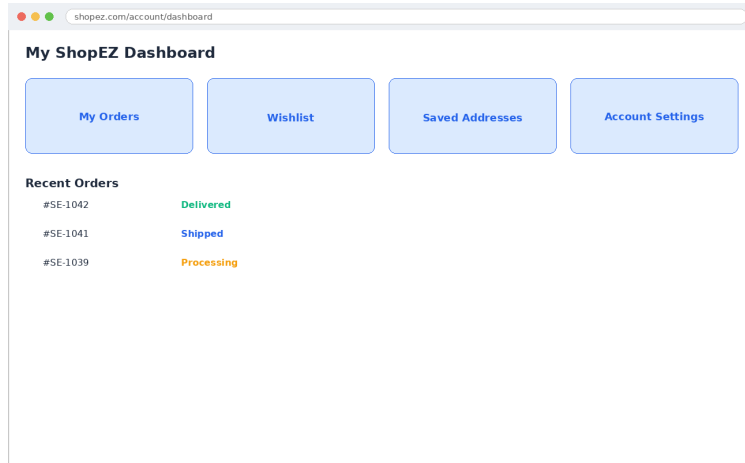
Admin Dashboard

Total Sellers 128	Total Customers 4,520	Orders Today 312	Revenue (₹) 1.2L
-----------------------------	---------------------------------	----------------------------	----------------------------

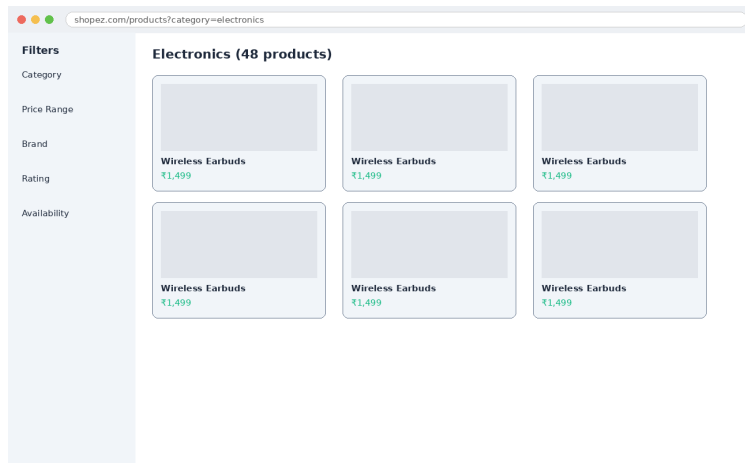
Recent Orders

#SE-1042	Delivered
#SE-1041	Shipped
#SE-1039	Processing
#SE-1038	Pending

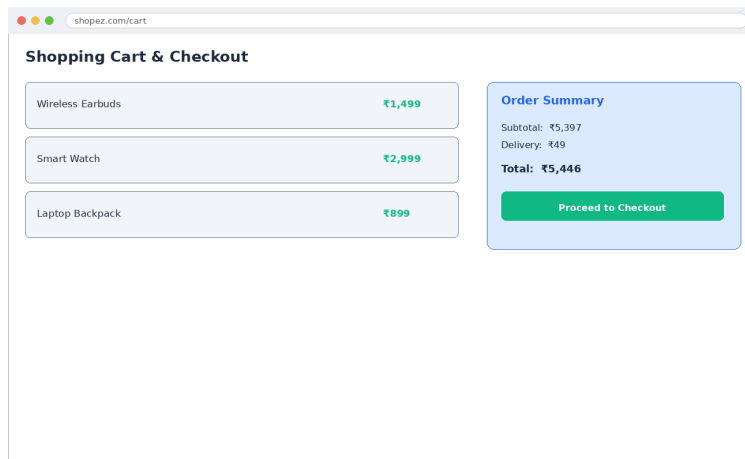
Customer Dashboard :



Product Listing :



Cart & Checkout Page :



10. Testing

Testing was performed to ensure system reliability and functionality.

Functional Testing

Verified all modules work according to requirements.

Authentication Testing

Verified registration, login, token validation, and role-based access.

Order Testing

Verified cart operations, checkout, order status updates, and cancellation workflows.

UI Testing

Verified responsiveness and usability across different screen sizes.

11. Screenshots or Demo

Project Demo Link :

Demo video / walkthrough available on request from the project author.

12. Known Issues

- Online payment gateway is not integrated (Cash on Delivery only).
- Email and SMS order notifications are not implemented.
- Product reviews and ratings cannot yet be submitted by customers.
- Wishlist functionality is not implemented.
- Mobile optimization can be improved further.

13. Future Enhancements

Future improvements planned for ShopEZ include:

- Payment Gateway Integration (Razorpay / Stripe)
- Product Reviews and Ratings System
- Wishlist / Save for Later
- AI-Based Product Recommendations
- SMS and Email Order Notifications
- Live Order Tracking
- Seller Analytics Dashboard
- Multi-Language Support
- Delivery Partner Integration
- Native Mobile App